Lecture 20: MIPS Registers Explained (Complete Register Set & Register Conventions)

🎯 Learning Objectives

By the end of this lecture, students will be able to:

- Understand what CPU registers are.
- Explain why registers are faster than RAM.
- Identify all **32 general-purpose MIPS registers**.
- Understand the purpose of each register category.
- Differentiate between temporary and saved registers.
- Understand caller-saved and callee-saved registers.
- Use registers correctly in MIPS Assembly programs.

---

1. Introduction

🎤 Teaching Script

First 30-Second SEO Phrase

Naturally mention:

> **"In today's Computer Organization and Assembly Language lecture, we'll learn the complete MIPS register set, including `$zero`, `$v`, `$a`, `$t`, `$s`, `$sp`, and `$ra`, and understand how each register is used in real MIPS Assembly programs."**

In the previous lecture, we learned about MIPS Architecture and saw that registers are one of the most important components of the CPU.

Today, we'll study these registers in detail.

Every MIPS instruction works with registers. Whether you're performing arithmetic, loading data from memory, passing arguments to a function, or returning a value, registers are involved.

Understanding how and when to use each register is the key to writing correct and efficient MIPS Assembly programs.

By the end of this lecture, you'll know the purpose of all 32 MIPS registers and the conventions programmers follow while using them.

Let's begin.

---

2. What is a Register?

A **register** is a **small, high-speed storage location inside the CPU**.

Registers temporarily store:

- Data
- Addresses
- Intermediate results
- Function parameters
- Return values

Unlike RAM, registers are located **inside the processor**, allowing the CPU to access them in a very short time.

---

3. Why Are Registers Faster Than RAM?

Comparison:

Feature	Registers	RAM

Location	Inside CPU	Outside CPU
Speed	Extremely Fast	Slower
Size	Very Small	Large
Purpose	Temporary storage	Main memory

Example

Suppose you want to add two numbers.

Instead of repeatedly reading from RAM:

RAM → CPU → RAM

The CPU loads the values into registers:

RAM → Register → ALU → Register

This reduces memory access and increases performance.

---

4. MIPS Register Overview

MIPS provides **32 general-purpose registers**, each **32 bits wide**.

Each register has:

- A **number** (`$0`–`$31`)

- A **name** (such as `$t0` or `$s1`)
- A **specific purpose** based on programming conventions.

---

5. Complete Register Table

Number	Name	Purpose
0	`$zero`	Always stores 0
1	`$at`	Assembler temporary
2–3	`$v0–$v1`	Return values
4–7	`$a0–$a3`	Function arguments
8–15	`$t0–$t7`	Temporary registers
16–23	`$s0–$s7`	Saved registers
24–25	`$t8–$t9`	Additional temporary registers
26–27	`$k0–$k1`	Reserved for the operating system/kernel

28	`$gp`	Global pointer
29	`$sp`	Stack pointer
30	`$fp` (or `$s8`)	Frame pointer
31	`$ra`	Return address

---

6. `$zero` Register

The `$zero` register is unique.

Its value is **always 0**.

Example:

add $t0, $zero, $zero

Result:

$t0 = 0

Attempting to modify `$zero` has no effect:

addi $zero, $zero, 5

The value of `$zero` remains **0**.

---

7. Return Value Registers (`$v0`, `$v1`)

Used to return results from functions.

Example:

add $v0, $t0, $t1

The result is stored in `$v0`.

`$v0` is also used to specify **system call codes**, which you'll learn later.

---

8. Argument Registers (`$a0–$a3`)

Used to pass parameters to functions.

Example:

add $a0, $zero, $t0

add $a1, $zero, $t1

Meaning:

Argument 1 → $a0

Argument 2 → $a1

A function can receive up to **four arguments** directly through these registers.

---

9. Temporary Registers (`$t0–$t9`)

Used for temporary calculations.

Example:

add $t0, $t1, $t2

Characteristics:

- Free to use.
- Values are **not preserved** across function calls.
- Best for intermediate calculations.

---

10. Saved Registers (`$s0–$s7`)

Used for values that must remain available after a function call.

Example:

add $s0, $t0, $t1

Characteristics:

- Preserve important data.
- If a function modifies an `$s` register, it must restore its original value before returning.

---

11. Difference Between `$t` and `$s`

Temporary (`$t`)	Saved (`$s`)
Temporary values	Long-term values
Not preserved	Must be preserved
Used for calculations	Used for important variables

Example

add $t0, $t1, $t2

Temporary result.

add $s0, $t1, $t2

Result that should remain available later.

---

12. Stack Pointer ($sp)

The Stack Pointer points to the **top of the stack**.

It is used for:

- Function calls
- Saving registers
- Local variables

Example:

addi $sp, $sp, -4

This allocates **4 bytes** on the stack.

---

13. Return Address Register (`$ra`)

When a function is called using:

jal function

The CPU automatically stores the return address in:

$ra

The function returns using:

jr $ra

---

14. Global Pointer (`$gp`)

The Global Pointer references frequently accessed global data.

Most beginner programs won't use it directly, but it plays an important role in larger applications.

---

15. Frame Pointer (`$fp`)

The Frame Pointer helps access:

- Local variables
- Function parameters

It remains stable throughout the execution of a function, making stack access easier.

---

16. Kernel Registers (`$k0`, `$k1`)

Reserved for the operating system.

Application programs should **not** use these registers.

---

17. Caller-Saved vs. Callee-Saved

Caller-Saved Registers

Examples:

$t0–$t9

The **calling function** is responsible for saving them if needed.

Callee-Saved Registers

Examples:

$s0–$s7

The **called function** must save and restore them before returning.

---

18. Register Naming Diagram

$zero Always 0

$v0-$v1 Return Values

$a0-$a3 Arguments

$t0-$t9 Temporary

$s0-$s7 Saved

$gp Global Pointer

$sp Stack Pointer

$fp Frame Pointer

$ra Return Address

Encourage students to memorize this grouping rather than all register numbers.

---

19. Real-Life Analogy

Imagine a student studying:

- **Registers** = Notebook on the desk (fast, immediately accessible)
- **RAM** = Bookshelf across the room (larger but slower)
- **Hard Disk** = Library storage room (very large but much slower)

A student keeps the information they're actively using in the notebook, just as the CPU keeps active data in registers.

---

20. Common Mistakes

Students often:

❌ Use `$zero` to store data.

✔ `$zero` is read-only and always contains 0.

❌ Use `$k0` and `$k1` in user programs.

✔ These are reserved for the operating system.

❌ Confuse `$t` and `$s` registers.

✔ Use `$t` for temporary values and `$s` for values that need to survive function calls.

🎤 Lecture Summary

Today, we explored the complete set of MIPS registers and learned how each category is used in Assembly Language programming.

We discussed the purpose of `$zero`, `$v`, `$a`, `$t`, `$s`, `$sp`, `$fp`, `$gp`, and `$ra`, and understood the difference between caller-saved and callee-saved registers. These conventions are essential for writing reliable and organized MIPS programs.

In the next lecture, we'll set up the MARS simulator, explore its interface, and write our **first MIPS Assembly program**, learning about sections such as `.data` and `.text`, labels, comments, and the basic structure of a MIPS program.

Thank you for watching, and I'll see you in the next lecture.
